S2S-10: Migration Scripts

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 10 | **Created:** April 2026 | **Last Updated:** 10/23/2026

Overview

Reusable scripts for SaaS-to-SaaS configuration export. They download Monaco, export the source tenant's configuration with a short-lived read-only token, and leave a timestamped export folder that you review, commit to Git, and **deploy directly to the target with `monaco deploy`**. As an option, the same scripts also package the export as a `.tar.gz` for the SaaS Upgrade Assistant (SUA) on the target — a field practice, not a documented SaaS-source path (§1). Both Bash (macOS/Linux/WSL) and PowerShell (Windows) versions are provided.

> **Why scripts instead of raw `monaco download`?** They handle platform detection, Monaco binary download, checksum verification, temporary token creation and revocation, and the export in a single command — and they leave the Monaco binary in place for the deploy step.

Table of Contents

1. Two Import Paths: Monaco Deploy or the SaaS Upgrade Assistant
2. Monaco Configuration Export (Bash)
3. Monaco Configuration Export (PowerShell)
4. Usage Notes
5. Post-Export Workflow

Prerequisites

Requirement	Details
API Token	Token with `apiTokens.write` scope on the source tenant — the scripts use it to create, then revoke, a short-lived export token
Shell (Option A)	Bash (macOS, Linux, or WSL) with `curl`, `jq`, `shasum` — plus `tar` for the optional SUA package
Shell (Option B)	PowerShell 5.1+ on Windows 10/11 — the optional SUA package uses the `tar.exe` that ships with Windows 10 1803+ and Windows 11
Network	HTTPS access to `github.com` (Monaco download), the source tenant URL, and — for the deploy — the target tenant URL
Target credentials	For the deploy: a target API token for classic configuration and settings, plus a platform token or OAuth client for platform configuration (documents, automations, buckets, segments, SLOs, OpenPipeline)

1. Two Import Paths: Monaco Deploy or the SaaS Upgrade Assistant

The export is the same either way. What differs is how it reaches the target:

Path	Import into the target	Status
A — Monaco deploy (default)	`monaco deploy <manifest> --environment <target>`	Documented (Monaco CLI commands (DT docs))
B — SaaS Upgrade Assistant (optional)	Upload a `.tar.gz` package to the SaaS Upgrade Assistant app on the target, preview, then deploy selectively	Field practice — not documented for a SaaS source

Why Path B is labelled field practice. The SUA documentation describes a Managed source: *"SaaS Upgrade Assistant imports your Dynatrace Managed environment configuration."* It does not describe a SaaS source. In the field, the SUA app on a SaaS target has accepted Monaco exports packaged in the format below, and it adds what `monaco deploy` lacks: the docs list *"preview changes"*, partial deployment (*"Choose which*

configurations you want to migrate and run partial deployment") and *"Automatically update dashboard owners"*. Rehearse Path B on a non-production target before relying on it, and keep Path A as the fallback.

Terraform remains the choice for IAM and for teams already on Terraform: `terraform-provider-dynatrace -export` from the source, `terraform apply` with target credentials (see **S2S-06** §4 for workflows and **AUTOM** for the provider).

SUA Package Format (Path B)

Run either script with the SUA option (§2, §3) and it writes this package beside the export folder:

Requirement	Detail
Format	`.tar.gz` — field reports say `.zip` is rejected
Metadata file	`exportMetadata.json` at the archive root
Export directory	`export/` subdirectory holding the Monaco download output

`exportMetadata.json`:

```
{
  "clusterUuid": "&lt;source-tenant-id&gt;",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "2.30.0",
  "exportTimestamp": "&lt;unix-ms&gt;",
  "environments": [
    {
      "name": "&lt;source-tenant-id&gt;",
      "uuid": "&lt;source-tenant-id&gt;"
    }
  ]
}
```

Directory layout inside the archive:

```
configurationExport-YYYY-MM-DD_HH-MM-SS/
├── exportMetadata.json
└── export/
    └── project/
        ├── alerting-profile/
        ├── auto-tag/
        ├── dashboard/
        └── ... (one folder per configuration type)
```

> **Field note:** raw `monaco download` output uploaded without this wrapper has been reported as rejected. The metadata fields mirror a Managed cluster export; the `productVersion` value is the one used in field-tested packages, not a documented requirement.
>
> **Sources:** SaaS Upgrade Assistant (DT docs) — *"SaaS Upgrade Assistant imports your Dynatrace Managed environment configuration"*, *"preview changes"*, *"Choose which configurations you want to migrate and run partial deployment"*, *"Automatically update dashboard owners"*; Monaco CLI commands (DT docs).

2. Monaco Configuration Export (Bash)

Downloads Monaco, exports all configuration from the source tenant, and stages it in a timestamped folder for `monaco deploy`. Set `SUA_PACKAGE=1` to also write the optional SaaS Upgrade Assistant package (§1, Path B).

> **Deprecation — Dynatrace API 1.348 (rollout start 10/12/2026, staged tenant rollout).** The API 1.348 changelog marks the whole `/apiTokens` endpoint family deprecated — *"The following endpoints are deprecated"*, covering `POST /apiTokens`, `POST /apiTokens/lookup` and the per-token `GET` / `PUT` / `DELETE`. No successor is named. Deprecated endpoints keep working during the deprecation period, so both scripts below (which mint a temporary export token with `POST /api/v2/apiTokens`) remain the working path — re-check the API 1.348 changelog (DT docs) for a successor before building new automation on that endpoint. Separately, once an environment opts into **Phase 3** of the upgrade to Latest Dynatrace, *"classic API token creation is disabled; all new integrations must use platform tokens"* (Best practices for upgrading App Observability API endpoints (DT docs)) — check your

> environment's upgrade phase before running these scripts, since each one mints a classic token.

Usage:

```
export ENV_TOKEN="dt0c01.XXXX..."
./monaco-export-migration.sh &lt;source-tenant-id&gt;

# Example
./monaco-export-migration.sh abc12345

# Also write the optional SUA package (Path B)
SUA_PACKAGE=1 ./monaco-export-migration.sh abc12345
```

Script:

```
#!/bin/bash
#
# Monaco Configuration Export for SaaS-to-SaaS migration
# Produces a timestamped export folder for review and `monaco deploy` to the
target.
# With SUA_PACKAGE=1 it also writes a .tar.gz for the SaaS Upgrade Assistant
(field practice).
#
# Usage: ENV_TOKEN=&lt;api-token&gt; [SUA_PACKAGE=1] ./monaco-export-
migration.sh &lt;tenantId&gt;

set -euo pipefail

MONACO_VERSION="2.30.0"   # pinned; check the releases page before reuse

# --- Argument validation ---
if [ $# -ne 1 ]; then
    echo "Usage: $0 &lt;tenantId&gt;"
    echo ""
    echo "Example: ENV_TOKEN=\$MY_TOKEN $0 abc12345"
    exit 1
fi

if [ -z "${ENV_TOKEN:-}" ]; then
    echo "Error: ENV_TOKEN environment variable is not set."
    echo "Create a token with apiTokens.write scope at:"
    echo "  https://$1.live.dynatrace.com/#settings/integration/apikeys"
    exit 1
fi

tenantId="$1"
echo "=== Monaco Export v${MONACO_VERSION} ==="
echo "Tenant: ${tenantId}"
echo ""

# --- Detect platform ---
OS=$(uname -s)
ARCH=$(uname -m)

case "${OS}-${ARCH}" in
    Darwin-arm64)  platform="darwin-arm64" ;;
    Darwin-x86_64) platform="darwin-amd64" ;;
    Linux-x86_64)  platform="linux-amd64" ;;
    Linux-aarch64) platform="linux-arm64" ;;
    Linux-i386)    platform="linux-386" ;;
    *)
        echo "Error: Unsupported platform ${OS}-${ARCH}"
```

```
        exit 1
        ;;
esac

echo "Platform: ${platform}"

# --- Download Monaco ---
binary_url="https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/download/v${MONACO_VERSION}/monaco-${platform}"
checksum_url="${binary_url}.sha256"

echo "Downloading Monaco v${MONACO_VERSION}..."
curl -sL -o "monaco-${platform}" "${binary_url}"
curl -sL -o monaco_checksum "${checksum_url}"

echo "Verifying checksum..."
if shasum --quiet -c monaco_checksum 2&gt;/dev/null; then
    echo "  Checksum verified"
else
    echo "  Checksum verification failed"
    rm -f "monaco-${platform}" monaco_checksum
    exit 1
fi

mv "monaco-${platform}" monaco
chmod +x monaco
echo "  Monaco v${MONACO_VERSION} ready"
echo ""

# --- Revoke the export token on exit (success or failure) ---
MONACO_TOKEN_ID=""
revoke_export_token() {
    if [ -n "${MONACO_TOKEN_ID}" ] &amp;&amp; [ "${MONACO_TOKEN_ID}" != "null"
]; then
        echo "Revoking temporary export token..."
        curl -s -o /dev/null -w "  DELETE apiTokens -&gt; HTTP %{http_code}\n"
-X DELETE \

"https://${tenantId}.live.dynatrace.com/api/v2/apiTokens/${MONACO_TOKEN_ID}" \
          -H "Authorization: Api-Token ${ENV_TOKEN}" \
          || echo "  Revoke failed: delete token ${MONACO_TOKEN_ID} manually"
    fi
}
trap revoke_export_token EXIT

# --- Create short-lived export token (read scopes only, expires in 24 h) ---
echo "Creating temporary export token..."
expiry=$(date -u -v+1d +%Y-%m-%dT%H:%M:%S 2&gt;/dev/null || date -u -d '+1
```

```
day' +%Y-%m-%dT%H:%M:%S)
token_body=$(jq -n --arg exp "${expiry}" '{
    name: "s2s-monaco-export-temp",
    expirationDate: $exp,
    scopes: [
      "attacks.read",
      "entities.read",
      "extensionConfigurations.read",
      "extensionEnvironment.read",
      "extensions.read",
      "geographicRegions.read",
      "javaScriptMappingFiles.read",
      "networkZones.read",
      "settings.read",
      "slo.read",
      "syntheticExecutions.read",
      "syntheticLocations.read",
      "DataExport",
      "ReadConfig",
      "ReadSyntheticData"
    ]
  }')

token_response=$(curl -s -X POST
"https://${tenantId}.live.dynatrace.com/api/v2/apiTokens" \
  -H "accept: application/json; charset=utf-8" \
  -H "Content-Type: application/json; charset=utf-8" \
  -H "Authorization: Api-Token ${ENV_TOKEN}" \
  -d "${token_body}")
MONACO_TOKEN=$(echo "${token_response}" | jq -r '.token // empty')
MONACO_TOKEN_ID=$(echo "${token_response}" | jq -r '.id // empty')

if [ -z "${MONACO_TOKEN}" ]; then
    echo "  Failed to create export token. Check ENV_TOKEN permissions
(apiTokens.write)."
    rm -f monaco monaco_checksum
    exit 1
fi
export MONACO_TOKEN
echo "  Export token created (expires ${expiry} UTC)"
echo ""

# --- Create manifest ---
cat &gt; manifest.yaml &lt;&lt;EOF
manifestVersion: 1.0

projects:
- name: saas
```

```
  path: saas/${tenantId}

environmentGroups:
- name: saas
  environments:
  - name: ${tenantId}
    url:
      value: https://${tenantId}.live.dynatrace.com
    auth:
      token:
        name: MONACO_TOKEN
EOF

# --- Run Monaco download ---
echo "Running Monaco download..."
echo "  This may take several minutes depending on configuration volume."
echo ""
./monaco download --environment "${tenantId}" --output-folder "${tenantId}"
echo ""

# The export token is no longer needed: revoke it now (the EXIT trap covers
failures)
revoke_export_token
MONACO_TOKEN_ID=""
unset MONACO_TOKEN
echo ""

# --- Stage the export for review and deploy ---
datetime=$(date +"%Y-%m-%d_%H-%M-%S")
export_dir="export-${tenantId}-${datetime}"
mv "${tenantId}" "${export_dir}"

# --- Optional: package for the SaaS Upgrade Assistant (Path B, field
practice) ---
sua_archive=""
if [ "${SUA_PACKAGE:-0}" = "1" ]; then
    sua_dir="configurationExport-${datetime}"
    mkdir -p "${sua_dir}/export"
    current_timestamp=$(($(date +%s) * 1000))
    cat &gt; "${sua_dir}/exportMetadata.json" &lt;&lt;EOF
{
  "clusterUuid": "${tenantId}",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "${MONACO_VERSION}",
  "exportTimestamp": "${current_timestamp}",
  "environments": [
    {
      "name": "${tenantId}",
```

```
      "uuid": "${tenantId}"
    }
  ]
}
EOF
    cp -R "${export_dir}/." "${sua_dir}/export/"
    tar -czf "${sua_dir}.tar.gz" "${sua_dir}"
    rm -rf "${sua_dir}"
    sua_archive="${sua_dir}.tar.gz"
fi

# --- Summary ---
echo ""
echo "=== Export Summary ==="
echo "Tenant:    ${tenantId}"
echo "Monaco:    v${MONACO_VERSION}"
echo "Output:    ${export_dir}/ (manifest.yaml + project/)"
if [ -n "${sua_archive}" ]; then
    echo "SUA:       ${sua_archive} (upload to the SaaS Upgrade Assistant on
the target)"
fi
echo ""

config_count=$(find "${export_dir}" -name "*.json" -o -name "*.yaml"
2&gt;/dev/null | wc -l | tr -d ' ')
echo "Configs exported: ~${config_count} files"
echo ""

echo "Next steps:"
echo "  1. Commit ${export_dir}/ to Git"
echo "  2. Add the target environment to ${export_dir}/manifest.yaml (S2S-05,
section 2)"
echo "  3. Remap entity IDs (S2S-05, section 4)"
echo "  4. ./monaco deploy ${export_dir}/manifest.yaml --environment target-
tenant --dry-run"
echo "  5. ./monaco deploy ${export_dir}/manifest.yaml --environment target-
tenant"
echo ""

# --- Cleanup (the monaco binary is kept for the deploy step) ---
rm -f monaco_checksum manifest.yaml

echo "Done."
```

3. Monaco Configuration Export (PowerShell)

Windows equivalent of the Bash script above. Produces the same timestamped export folder for `monaco deploy`. Add `-PackageForSUA` to also write the optional SaaS Upgrade Assistant package (§1, Path B).

Usage:

```
$env:ENV_TOKEN = "dt0c01.XXXX..."
.\Monaco-Export-Migration.ps1 -TenantId abc12345

# Also write the optional SUA package (Path B)
.\Monaco-Export-Migration.ps1 -TenantId abc12345 -PackageForSUA
```

Script:

```
#Requires -Version 5.1
&lt;#
.SYNOPSIS
    Monaco Configuration Export for SaaS-to-SaaS migration
.DESCRIPTION
    Downloads Monaco, exports all configuration from a source tenant,
    and stages the result in a timestamped folder for monaco deploy.
.PARAMETER TenantId
    The Dynatrace tenant identifier (e.g., abc12345)
.PARAMETER PackageForSUA
    Also write a .tar.gz for the SaaS Upgrade Assistant (field practice; not
documented for a SaaS source)
.EXAMPLE
    $env:ENV_TOKEN = "dt0c01.XXXX..."
    .\Monaco-Export-Migration.ps1 -TenantId abc12345
#&gt;
param(
    [Parameter(Mandatory)]
    [string]$TenantId,
    [switch]$PackageForSUA
)

$ErrorActionPreference = "Stop"
$MonacoVersion = "2.30.0"   # pinned; check the releases page before reuse

# --- Validate token ---
if (-not $env:ENV_TOKEN) {
    Write-Error "ENV_TOKEN environment variable is not set.`nCreate a token
with apiTokens.write scope at:`n
https://$TenantId.live.dynatrace.com/#settings/integration/apikeys"
}

Write-Host "=== Monaco Export v$MonacoVersion ==="
Write-Host "Tenant: $TenantId"
Write-Host ""

# --- Download Monaco (Windows amd64) ---
$platform = "windows-amd64"
$binaryUrl = "https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/download/v$MonacoVersion/monaco-$platform.exe"
$checksumUrl = "$binaryUrl.sha256"

Write-Host "Downloading Monaco v$MonacoVersion..."
Invoke-WebRequest -Uri $binaryUrl -OutFile "monaco.exe" -UseBasicParsing
Invoke-WebRequest -Uri $checksumUrl -OutFile "monaco_checksum" -
UseBasicParsing
```

```
Write-Host "Verifying checksum..."
$expectedHash = (Get-Content monaco_checksum -Raw).Split()[0].Trim()
$actualHash = (Get-FileHash -Algorithm SHA256 monaco.exe).Hash.ToLower()
if ($actualHash -ne $expectedHash) {
    Remove-Item -Force monaco.exe, monaco_checksum
    Write-Error "Checksum verification failed"
}
Write-Host "  Checksum verified"
Write-Host "  Monaco v$MonacoVersion ready"
Write-Host ""

# --- Create short-lived export token (read scopes only, expires in 24 h) ---
Write-Host "Creating temporary export token..."
$expiry = (Get-Date).ToUniversalTime().AddDays(1).ToString("yyyy-MM-
ddTHH:mm:ss")
$tokenBody = @{
    name           = "s2s-monaco-export-temp"
    expirationDate = $expiry
    scopes         = @(
        "attacks.read", "entities.read", "extensionConfigurations.read",
        "extensionEnvironment.read", "extensions.read",
"geographicRegions.read",
        "javaScriptMappingFiles.read", "networkZones.read", "settings.read",
        "slo.read", "syntheticExecutions.read", "syntheticLocations.read",
        "DataExport", "ReadConfig", "ReadSyntheticData"
    )
} | ConvertTo-Json

$headers = @{
    "Authorization" = "Api-Token $($env:ENV_TOKEN)"
    "Content-Type"  = "application/json; charset=utf-8"
}

$response = Invoke-RestMethod `
    -Uri "https://$TenantId.live.dynatrace.com/api/v2/apiTokens" `
    -Method Post -Headers $headers -Body $tokenBody

if (-not $response.token) {
    Remove-Item -Force monaco.exe, monaco_checksum -ErrorAction
SilentlyContinue
    Write-Error "Failed to create export token. Check ENV_TOKEN permissions."
}
$env:MONACO_TOKEN = $response.token
$tokenId = $response.id
Write-Host "  Export token created (expires $expiry UTC)"
Write-Host ""

# --- Create manifest ---
```

```
@"
manifestVersion: 1.0

projects:
- name: saas
  path: saas/$TenantId

environmentGroups:
- name: saas
  environments:
  - name: $TenantId
    url:
      value: https://$TenantId.live.dynatrace.com
    auth:
      token:
        name: MONACO_TOKEN
"@ | Set-Content -Path "manifest.yaml" -Encoding UTF8

# --- Run Monaco download ---
Write-Host "Running Monaco download..."
Write-Host "  This may take several minutes depending on configuration
volume."
Write-Host ""
.\monaco.exe download --environment $TenantId --output-folder $TenantId
$downloadExit = $LASTEXITCODE
Write-Host ""

# --- Revoke the export token (also when the download failed) ---
Write-Host "Revoking temporary export token..."
try {
    Invoke-RestMethod `
        -Uri "https://$TenantId.live.dynatrace.com/api/v2/apiTokens/$tokenId"
`
        -Method Delete -Headers $headers | Out-Null
    Write-Host "  Export token revoked"
} catch {
    Write-Warning "Revoke failed: delete token $tokenId manually (it expires
$expiry UTC regardless)"
}
Remove-Item Env:\MONACO_TOKEN -ErrorAction SilentlyContinue
if ($downloadExit -ne 0) {
    Write-Error "monaco download failed with exit code $downloadExit"
}
Write-Host ""

# --- Stage the export for review and deploy ---
$datetime = Get-Date -Format "yyyy-MM-dd_HH-mm-ss"
$exportDir = "export-$TenantId-$datetime"
```

```
Rename-Item -Path $TenantId -NewName $exportDir

# --- Optional: package for the SaaS Upgrade Assistant (Path B, field
practice) ---
$suaArchive = $null
if ($PackageForSUA) {
    $suaDir = "configurationExport-$datetime"
    New-Item -ItemType Directory -Path "$suaDir\export" -Force | Out-Null
    $currentTimestamp = [long]
([DateTimeOffset]::UtcNow.ToUnixTimeMilliseconds())
    @"
{
  "clusterUuid": "$TenantId",
  "productVersion": "1.305.0.20260331-000000",
  "monacoVersion": "$MonacoVersion",
  "exportTimestamp": "$currentTimestamp",
  "environments": [
    {
      "name": "$TenantId",
      "uuid": "$TenantId"
    }
  ]
}
"@ | Set-Content -Path "$suaDir\exportMetadata.json" -Encoding UTF8
    Copy-Item -Path "$exportDir\*" -Destination "$suaDir\export\" -Recurse -
Force
    tar -czf "$suaDir.tar.gz" $suaDir
    Remove-Item -Recurse -Force $suaDir
    $suaArchive = "$suaDir.tar.gz"
}

# --- Summary ---
Write-Host ""
Write-Host "=== Export Summary ==="
Write-Host "Tenant:    $TenantId"
Write-Host "Monaco:    v$MonacoVersion"
Write-Host "Output:    $exportDir\ (manifest.yaml + project\)"
if ($suaArchive) {
    Write-Host "SUA:       $suaArchive (upload to the SaaS Upgrade Assistant
on the target)"
}
Write-Host ""

$configCount = (Get-ChildItem -Path $exportDir -Recurse -Include *.json,
*.yaml).Count
Write-Host "Configs exported: ~$configCount files"
Write-Host ""
```

```
Write-Host "Next steps:"
Write-Host "  1. Commit $exportDir\ to Git"
Write-Host "  2. Add the target environment to $exportDir\manifest.yaml (S2S-
05, section 2)"
Write-Host "  3. Remap entity IDs (S2S-05, section 4)"
Write-Host "  4. .\monaco.exe deploy $exportDir\manifest.yaml --environment
target-tenant --dry-run"
Write-Host "  5. .\monaco.exe deploy $exportDir\manifest.yaml --environment
target-tenant"
Write-Host ""

# --- Cleanup (monaco.exe is kept for the deploy step) ---
Remove-Item -Force monaco_checksum, manifest.yaml -ErrorAction
SilentlyContinue

Write-Host "Done."
```

> **Note:** the optional SUA package uses the `tar.exe` included with Windows 10 version 1803+ and Windows 11. On older systems, install 7-Zip and replace the `tar` line with:
> `powershell > & "C:\Program Files\7-Zip\7z.exe" a -ttar "$suaDir.tar" $suaDir > & "C:\Program Files\7-Zip\7z.exe" a -tgzip "$suaDir.tar.gz" "$suaDir.tar" > Remove-Item "$suaDir.tar" >`

4. Usage Notes

Bash (macOS / Linux / WSL)

- Save the Bash script to a file (e.g., `monaco-export-migration.sh`) and run `chmod +x monaco-export-migration.sh`
- Requires `curl`, `jq` and `shasum`

PowerShell (Windows)

- Save the PowerShell script to a file (e.g., `Monaco-Export-Migration.ps1`)
- If execution policy blocks the script, run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Both Scripts

- The script downloads Monaco automatically — no pre-installation required — and keeps the binary for the deploy step
- A short-lived export token (`s2s-monaco-export-temp`) is created with read scopes only, expires after 24 hours, and is revoked as soon as the download finishes — also when it fails
- The export token is a classic API token, so the download covers classic configuration and Settings 2.0. Platform configuration (documents, automations, buckets, segments, SLOs, OpenPipeline) needs a platform token or OAuth client — Monaco's `--platform-token` flag takes the *"Platform token environment variable"*, and `--oauth-client-id` / `--oauth-client-secret` name an OAuth client's variables (**S2S-04** §3)
- Monaco is pinned to v2.30.0 (released 09/23/2026); check the releases page before reuse and bump `MONACO_VERSION` / `$MonacoVersion`
- Output is a folder `export-<tenant>-<timestamp>/` holding the Monaco manifest and the `project/` configuration
- With `SUA_PACKAGE=1` (Bash) or `-PackageForSUA` (PowerShell), the script also writes `configurationExport-<timestamp>.tar.gz` for the SaaS Upgrade Assistant (§1, Path B) — the export folder is kept either way, so Path A stays available

> **Sources:** monaco download flags, v2.30.0 (Dynatrace GitHub).

Multi-Source Consolidation

When consolidating multiple source tenants into a single target, run the script once per source tenant:

```
# Source 1
ENV_TOKEN=$SOURCE1_TOKEN ./monaco-export-migration.sh tenant-id-1

# Source 2
ENV_TOKEN=$SOURCE2_TOKEN ./monaco-export-migration.sh tenant-id-2
```

Deploy (or import) Source 1 first, validate, then Source 2. Sequential migration isolates issues to a single source at a time. See **S2S-02** for the multi-source consolidation pattern.

5. Post-Export Workflow

After running the export script:

Step	Action	Reference
1	Commit the export folder to Git	The reviewed baseline for everything that follows
2	Add the target environment to the export's `manifest.yaml`	**S2S-05 §2: Monaco Deploy Workflow**
3	Audit for entity ID references	Search exported config for `HOST-`, `SERVICE-`, `PROCESS_GROUP-` patterns
4	Remap entity IDs to tag-, segment- or dimension-based filters	**S2S-05 §4: Entity ID Remapping**
5	`monaco deploy <manifest> --environment <target> --dry-run`	Checks structure and references only — it does not contact the target
6	`monaco deploy <manifest> --environment <target>` (per project, for phased deployment)	**S2S-05 §1: Configuration Deployment Order**
7	Validate data flow after each wave	**S2S-05 §8: Wave Execution and Validation**

Path B: Import Through the SaaS Upgrade Assistant (Optional)

Use this only after rehearsing it on a non-production target (§1).

Step	Action	Reference
1	Remap entity IDs in the export **before** packaging, then re-run packaging	**S2S-05 §4: Entity ID Remapping**
2	Install the SaaS Upgrade Assistant app on the target and upload `configurationExport-<timestamp>.tar.gz`	SaaS Upgrade Assistant (DT docs)

Step	Action	Reference
3	Preview the changes and select which configurations to deploy	The docs' *"preview changes"* and partial deployment
4	Deploy in waves per the configuration deployment order	**S2S-05 §1: Configuration Deployment Order**
5	Validate data flow after each wave; fall back to Path A for anything the assistant rejects	**S2S-05 §8: Wave Execution and Validation**

Path A: Deploy to the Target

```
export DT_TARGET_URL="https://&lt;target-env-id&gt;.live.dynatrace.com"
export DT_TARGET_TOKEN="dt0c01.xxx..."   # the manifest names this variable;
it never holds the token

./monaco deploy export-&lt;tenant&gt;-&lt;timestamp&gt;/manifest.yaml --
environment target-tenant --dry-run
./monaco deploy export-&lt;tenant&gt;-&lt;timestamp&gt;/manifest.yaml --
environment target-tenant
```

A clean dry-run is not a guarantee: Monaco's flag help says a dry-run *"can not validate the content of JSON payloads. After a successful dry-run, deployments may still fail with Dynatrace API errors if the content of JSONs is not valid."* Deploy to a non-production target first.

> **Sources:** monaco deploy flags, v2.30.0 (Dynatrace GitHub).

When NOT to Use These Scripts

Scenario	Use Instead
You only need specific config types	`monaco download --only-settings`, `--settings-schema <schema>` or `--api <api>`
You want ongoing config management	Terraform with state management

Scenario	Use Instead
The source is a Managed environment	The SaaS Upgrade Assistant (see the M2S series)
You need IAM migration	Terraform (`dynatrace_iam_*`), or `monaco account download` / `deploy` with an OAuth client

Script vs. Manual Monaco

Feature	Script	Manual `monaco download`
Monaco installation	Automatic (downloads + verifies checksum)	Manual pre-installation required
Export token	Auto-created with read scopes, 24 h expiry, revoked after download	Manual token creation and revocation
Output	Timestamped export folder, ready for `monaco deploy`	Whatever `--output-folder` you choose
SUA package	Optional `.tar.gz` with `exportMetadata.json` (`SUA_PACKAGE=1` / `-PackageForSUA`)	Manual packaging
Cleanup	Automatic (removes temp files, keeps the binary)	Manual cleanup
Cross-platform	Bash + PowerShell versions	Single platform

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.